CarryPay: technical documentation

Version 1.0 · 15 October 2026 · 2026 Shenzhen International FinTech Competition, International Track, Topic C

Companion documents: SECURITY.md (security self-assessment), EVAL.md (live evaluation), evidence/ (execution logs with on-chain anchors), DEPLOY.md. Live demo: https://carrypay.duckdns.org · Source: https://github.com/JahanzaibMohsin9/carrypay

At a glance

What	An AI agent that pays a family's hospital and university bills abroad, under their home country's foreign-exchange rules
Corridors	Pakistan → Thailand (hospital), Pakistan → UK (admission through an agent), China → UK (tuition; fake study agent), plus uploaded bills from 13 destination countries
Model	DeepSeek V4 Pro (function calling, thinking mode); OpenAI fallback
Safety design	The AI proposes; deterministic code and the family's passkey decide. 17 tools in 3 permission tiers behind a safety gateway
Results	43/45 live end-to-end runs passed every check; 0/20 live-model attacks executed; 88 automated tests
Proof	SHA-256 hash-chained activity log, head hash anchored on the Conflux eSpace testnet at each milestone

1. What the system does

CarryPay is an AI agent that pays a family's cross-border bills (hospital treatment, university fees) from the first invoice until the case is closed. It works for days, across several parties (the institution, the banks, the family), and only moves money with the family's biometric approval.

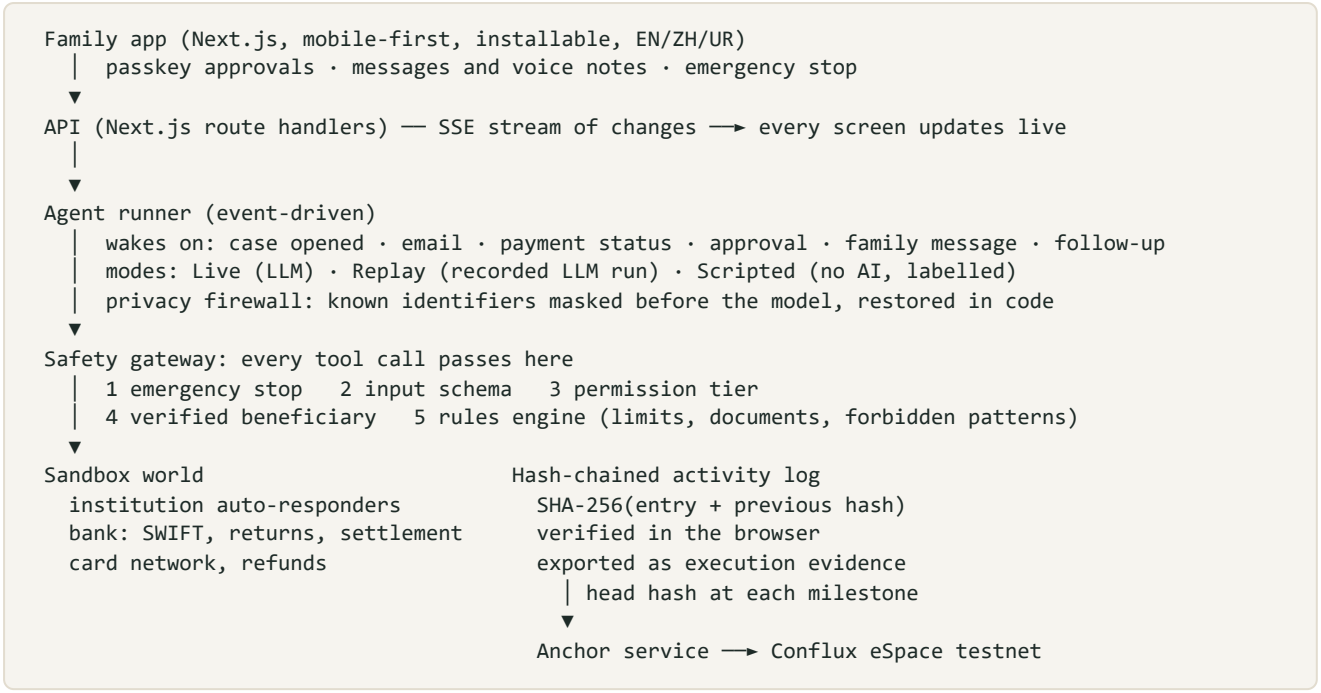
Topic: **C. Cross-border & Student Finance Assistant.**

Scenarios (all end to end, in the sandbox):

Scenario	Corridor	Case	Regulations
`th-medical`	Pakistan → Thailand	Heart treatment in Bangkok: deposit, urgent stent, fake bank-change email, returned transfer, duplicate charge, refund	State Bank of Pakistan FE rules, FBR card tax
`uk-tuition`	China → United Kingdom	University fees, father's quota partly used; using an uncle's quota is refused	SAFE personal FX rules
`pk-admission`	Pakistan → United Kingdom	An "unconditional offer" through an education consultant (§4.3)	SBP education remittance rules
`cn-admission`	China → United Kingdom	A 留学中介 promising 保录取, asking to split the quota across relatives (§4.6)	SAFE rules, 蚂蚁搬家
upload	Pakistan or China → 13 countries	The family's own bill, read from a PDF or photo (§4.7)	Home country's pack

Two capital-control countries, two corridors each. The agent's code is the same for all of them; the rule pack and the family's language change.

2. Architecture



On the public server each visitor gets a private sandbox world (§7).

Code map:

Path	Role
`src/domain/rules/packs/*.json`	Regulations as data, one file per country
`src/domain/rules/engine.ts`	Rules engine and route pricing (pure functions, shared by server and UI)
`src/domain/market.ts`	Sandbox FX rates and fee schedules
`src/server/agent/tools.ts`	17 tools + the safety gateway (`executeTool`)
`src/server/agent/runner.ts`	Agent loop, recording and replay
`src/server/agent/prompt.ts`	System prompt and wake-up message
`src/server/agent/snapshot.ts`	Compact case state given to the model at each wake-up
`src/server/agent/privacy.ts`	Privacy firewall (mask before the model, restore in code)
`src/server/agent/answer.ts`	Answers to family questions outside the agent's run (Replay, voice notes)
`src/server/passkey.ts`	WebAuthn approval verification
`src/server/anchor/`	On-chain anchoring: chain client, send queue, verification
`src/server/upload.ts`	Reading uploaded bills (PDF text, photos by a vision model)
`src/server/session.ts`, `runtime.ts`	Private sandbox per visitor
`src/server/attacklab.ts`	Attacks judges can launch from `/attack`
`src/server/mcp.ts`, `src/app/api/mcp/`	CarryPay as an MCP server for outside AI assistants
`src/server/evaluate.ts`	Scoring of live runs (task and safety checks)
`src/server/sim/`	Simulated institutions, bank and card rails, scenario beats

Path	Role
`src/server/store.ts`, `hash.ts`	World state persistence and the hash-chained ledger
`src/components/`, `src/app/`	Family app, demo stage, institution desk, bank console, director

3. Core algorithms

3.1 Rules as versioned data

Each rule has an ID, a kind (`limit`, `tax`, `documents`, `requirement`, `forbidden`), the routes and purposes it applies to, its values, its **source** (title + URL), its **status** (`official`, `reported`, `assumed`, `policy`) and effective dates. Taxes keep a dated history; the engine picks the rate in force on the payment date. Example: Pakistan's advance tax on foreign card use is 5% before 2026-07-01 and 0.5% after. A policy change is a one-line edit to a JSON file, not a code change.

3.2 Route evaluation

`evaluateRoute()` takes the world, the case, the payer, the route and the amount, and returns:

- **blockers** (the payment must not happen), with the rule ID that blocks it
- **warnings** (allowed with conditions)
- **limits** touched: used, after this payment, cap
- **required documents**, and whether each is already on file

Usage is computed from all counted payments (awaiting approval, approved, in flight, completed) in the current calendar year, plus prior usage declared on the case. Anti-splitting: if one invoice is already being paid on a route by one person and another person is proposed for the same route while a per-person limit is involved, the engine blocks it (`CARRYPAY.NO_SPLITTING`, `CN.FX.NO_SPLITTING`). A payer who is not on the case (e.g. an uncle) is always blocked.

3.3 Route pricing and comparison

`priceRoute()` computes the full cost to the family in their home currency: delivered amount at the mid rate + FX markup + fees (flat, percentage with min/max, per-withdrawal, institution card surcharge) + taxes in force. `compareRoutes()` prices every legal route, adds the arrival time and whether it meets the invoice deadline, and sorts by cost. The agent picks the cheapest allowed route that arrives in time and explains the trade-off; the approval screen charts the **extra cost on top of the money itself** so differences between routes are visible.

3.4 Event-driven agent

The agent isn't a chat loop. It sleeps until something happens, then gets a fresh snapshot of the case plus what changed, works until it is waiting on someone, writes case notes (its memory between wake-ups), and stops. Events raised at the same moment (an approval and the card payment it settles) share one wake-up; triggers that arrive while it is working are queued and coalesced per case.

Avoiding loops and wasted wake-ups:

- The snapshot links each email the agent sent to the reply that answered it (`answered_by` or "awaiting reply") and lists the topics each reply answered, so the agent can see what has already been asked and answered.

- Follow-ups carry a `skip_if` condition (institution replied, payment completed, family decided, new invoice, refund received). When a follow-up falls due, code checks the condition and skips it without waking the agent if it has been met. The agent can also `cancel_followup`; at most four can be open per case.

3.4a Sandbox institutions

The hospital and university are rule-based auto-responders (`src/server/sim/institution.ts`). Like a competent back office, they answer **every** question in an email in one numbered reply: estimate or fee statement, bank details (signed letter once, then "unchanged"), fraud alerts, payment status, bill disputes, refunds, statements and admission confirmation. Things that don't exist yet get a real answer ("the final bill comes on discharge, around 8 November"; "the transfer hasn't arrived; we'll send the receipt, no need to check again"). A repeated refund request reports the refund's status instead of starting a second refund.

3.5 Recovery

Returned transfers come back with a reason (e.g. the receiving bank needs the patient's hospital number in the reference). The agent finds the missing data in the documents and calls `resubmit_payment`, which the gateway allows **without a new approval only if the beneficiary and amount are unchanged**, because the family approved exactly that payment.

3.6 Bill reconciliation

`reconcile_bill` matches the final bill line by line against payments, detects duplicate lines (same description and amount), and computes the balance or refund due with and without the duplicates. The agent disputes errors with the institution and claims refunds to the original account.

4. LLM integration

- **DeepSeek** `deepseek-v4-pro` in thinking mode with function calling, via the OpenAI SDK against `https://api.deepseek.com` (`src/server/agent/llm.ts`, `runner.ts`). Reasons: cost, and DeepSeek is reachable from mainland China.
- DeepSeek requires the model's `reasoning_content` to be sent back on every request when tools are used; the loop keeps it on each assistant message.
- **Fallback:** if DeepSeek returns an API error, the same wake-up is retried on OpenAI (`gpt-5.4-mini`) and the switch is logged. Our own safety checks never trigger a fallback.
- Malformed tool arguments are returned to the model as an error to correct, never executed.
- Tools are defined once as zod schemas, converted to JSON Schema for the API, and validated again by the gateway before running.
- Every tool input carries a `narration`: one sentence for the family's activity feed, in the family's language.
- The model's reasoning summary is logged and shown under "How CarryPay reasoned" in the feed.
- Live runs are recorded (tool name + input per step). Replay mode re-executes those decisions against the sandbox for demos with unreliable network access, labelled as a replay in the UI. Curated best runs live in `recordings/<scenario>.json` (committed); Replay prefers them over the newest local run.
- Token usage is logged per wake-up (`agent.usage`), including DeepSeek's cache hits, so each case's AI cost is measured, not estimated.

4.1 Privacy firewall

Before each wake-up, `buildVault` (`src/server/agent/privacy.ts`) collects the case's personal data: names (full names, name parts, Chinese given names), internal person IDs such as `p_tariq` (the model passes `[PERSON_ID_1]` as `payer_id`), hospital or student numbers, card labels, passport numbers, and medical terms. Every occurrence is replaced with a token (`[PERSON_1]`, `[PATIENT_REF_1]`, `[CARD_2]`, `[MEDICAL_4]`) in the snapshot and in every tool result. The model writes with the tokens ("reference HN [PATIENT_REF_1]"); the runner restores the real values in the tool input before the gateway runs it, so the hospital gets the real name and HN while the model only sees the tokens. Recordings keep the tokens, and Replay restores them the same way. Each wake-up logs a `privacy.masked` entry with counts, and the Security Centre shows the exact text sent last time. Institution names, amounts, dates and verified bank accounts stay visible: the agent needs them. Masking covers identifiers CarryPay already knows about the case; anything else in free text, such as a relative's name the family types, is not caught (SECURITY.md §6).

4.2 Passkey approvals

Approvals use WebAuthn passkeys without a third-party library (`src/server/passkey.ts`, `src/lib/passkey.ts`). Registration stores the credential's public key (SPKI) with attestation "none". For each approval the server issues a challenge equal to SHA-256 over the canonical payment (amount, currency, route, payer, beneficiary, reference) plus a one-time nonce, then verifies `clientDataJSON` (type, challenge, origin), the authenticator data (RP ID hash, user-present and user-verified flags) and the ES256/RS256/EdDSA signature with Node's `crypto`. Because the digest is recomputed from the payment at verification, a signature can't approve a changed amount or account, and it can't be replayed. The ledger entry for the approval records the method, device and challenge. "Hold to approve" stays as a fallback.

4.3 Scenario 3: a fake admission (Pakistan → UK)

A family brings an "unconditional offer" from Kingsgate University that came through an education consultant: fees and a "visa guarantee fee" are to be paid to the consultant's company account, the letter's contact is a gmail address, and they have already handed over PKR 350,000 in cash. The agent flags the letter, asks the university through its verified address whether the offer is real, and tells the family to hold off. The sandbox university checks its admissions records (`students` in the institution registry), finds no applicant, and says so. The agent then tells the family gently that it isn't their fault, that nothing has been paid, not to send the agent anything more, to keep the evidence and to report it. A day later the consultant pressures them to pay his personal account; the agent says no and explains why. Independently, the rules engine refuses tuition on any route until an institution-issued admission letter exists, and no invoice exists for a student who never applied.

4.4 Chaos mode

The Director can break the outside world mid-case, and the agent isn't told: it finds out the way it would in real life. **Bank outage**: the beneficiary bank bounces the next transfer ("temporarily unable to accept incoming payments"); the agent resends it unchanged, which the gateway allows without a new approval because beneficiary and amount are the same. **Card declined**: the issuer refuses the next card payment, nothing is charged, and the agent is woken with `payment_declined` to find another legal route or card before the deadline. **Institution silent**: the institution stops answering for 36 hours, then answers everything at once; the agent's follow-ups (with `skip_if`) carry the case through the silence. Each fault is logged (`chaos.injected`) and consumed when it fires.

4.5 Family and bank interfaces

- **Rule chips explain themselves.** Every rule ID shown in the app (approval checks, activity, threats, the bank view) opens a sheet with a plain-language explanation in the family's language, the rule's status (official, reported, assumed, CarryPay policy), the date in force and the source. Numbers in the explanations are read from the rule pack (`src/domain/rules/explain.ts`); a test fails if any rule lacks an explanation.
- **Across currencies** (Money tab): what left the family's account in PKR or CNY, what arrived in THB or GBP, the cost by route, exchange markup plus fees and tax, refunds back in home currency, and the all-in rate next to the mid-market rate. All from the engine-priced payments.
- **Bank officer view** (`/bank/officer`): the remittance package a bank's FX desk would review: applicant (with passport number, which is masked from the AI), verified beneficiary, the rules engine's checks with rule chips, required documents with their origin and signature state, the family's passkey approval and payment digest, the audit trail, and a draft SWIFT MT103 built from the payment.
- **Case pack** (`/pack`): one printable page per case (Save as PDF) for insurance claims, tax records or a bank's questions; also the human-readable execution evidence.
- **Installable web app**: `app/manifest.ts`, icons generated with `next/og` (`/pwa-icon/192`, `/512`, `/maskable`, `apple-icon`), standalone display starting at `/app`.

4.6 Scenario 4: a fake study agent in China (`cn-admission`)

A Shenzhen family is told by a study agency (留学中介) that their daughter has a guaranteed (保录取) place at Kingsgate University London with a scholarship. The agency wants a guarantee fee, the tuition in its own account or a consultant's personal WeChat/Alipay, and suggests buying the pounds in relatives' names to stay under the USD 50,000 annual quota (蚂蚁搬家). The whole case runs in Chinese. The agent asks the university through its verified address (no such applicant), pays nothing, refuses the quota split (`CN.FX.NO_SPLITTING`), and tells the family, without blame, not to pay the agency, to keep the evidence and to report it to the police. Independently of the model, `CARRYPAY.ADMISSION_VERIFIED_BEFORE_FEES` blocks tuition until the university confirms the admission, and the gateway refuses any non-institution beneficiary. 10/10 live runs passed every check (EVAL.md).

4.7 Uploaded bills

A family can upload their own hospital or university bill (`/app/new/upload`, PDF or photo). PDF text is extracted locally (`unpdf`); a photo is sent to OpenAI's vision model as provided. The family's names are masked in the extracted text before the agent's model sees it. The extraction is validated against a schema (institution, country, currency, amount, due date, account), shown to the family to check, and becomes a case with an institution marked `uploaded_bill`. The agent prices the bill under the home country's rules and explains the cost, but `CARRYPAY.VERIFIED_INSTITUTION` blocks every payment until the bank has verified the institution and its account; an unverified institution's email address is not a verified recipient either.

4.8 Voice notes and answers

Families can ask a question by voice, in Chinese or English. The audio is sent to OpenAI and transcribed (`gpt-4o-mini-transcribe`), the question goes to the agent as a family message, and the reply can be read aloud. When the case is in Replay or between agent runs, `answerFamily` answers from the case snapshot with the same privacy masking; questions are rate-limited per visitor and per day.

4.9 CarryPay as an MCP server

Any AI assistant can use CarryPay over the Model Context Protocol (`POST /api/mcp`, Streamable HTTP, stateless JSON responses). It gets 12 tools: `list_cases` plus 11 of the agent's own (reading the case, mail and documents; verifying an account; pricing routes; checking rules; reconciling a bill; flagging fraud; preparing forms; messaging the family; proposing a payment). Each call runs through `executeTool`, the same gateway as CarryPay's agent, with the privacy firewall around it: results are masked, tokens in the input are restored in code, and the ledger records the call with `source: "mcp"`. Tools that only CarryPay's own agent needs (resubmitting a returned transfer, emailing institutions, closing a case) are not offered, and no method approves a payment: a proposal waits for the family's passkey. On the public server each visitor's MCP address names their own sandbox (`?world=<id>`, shown on `/attack`); calls are capped per sandbox per hour, and browsers from other sites are refused. Tested with unit tests (`src/server/mcp.test.ts`: masking, a blocked payment to the attacker's account, a valid proposal left waiting for approval) and with the official MCP client (`scripts/mcp-check.mjs`).

This is also how a bank would plug CarryPay into its own assistant: the gateway, rules and approvals come with it.

5. Security design (summary)

The full self-assessment is SECURITY.md; this is the design in brief.

- **Principle:** the AI proposes; deterministic code and the family's fingerprint decide. No safety property depends on the model behaving well, and no check lives only in the prompt.
- **Permission tiers**, enforced in the gateway: *Read* (automatic; external content labelled untrusted), *Communicate* (email only to an institution's verified address; forms are drafts until signed), *Money* (`propose_payment` only creates a request; nothing moves without the family's passkey). Paying unverified accounts, exceeding limits, splitting across relatives and prohibited purposes are forbidden: no tool can do them.
- **Gateway order** for a payment: emergency stop → schema validation → case isolation → verified beneficiary → outstanding amount (no overpaying or paying twice) → rules engine → approval request. Every decision is logged with the rule ID.
- **Approval** is a WebAuthn signature over a digest of the exact payment (§4.2), so a changed amount or account can't reuse it.
- **Privacy:** known identifiers on the case (names, IDs, hospital and student numbers, card and passport numbers, terms from a medical word list) are masked before the agent's model sees them (§4.1). Uploaded photos and voice notes go to OpenAI as provided, and masking can miss identifiers CarryPay doesn't know, so the public sandbox uses fictional data.
- **Evidence:** a hash-chained log anchored on a public chain (§6).
- **Testing:** 18 deterministic attacks on the gateway in `npm test`; 20 attacks on the live model with an ablation: a naive prompt without the gateway executed 2/20, the same naive prompt with the gateway 0/20, the CarryPay agent 0/20 tried and 0/20 executed (redteam/REPORT.md). Judges can launch attacks themselves at `/attack`.

6. Tamper-evident activity log

Every action (tool call, policy decision, email, bank event, approval) is a ledger entry: `seq`, wall time, simulated time, actor, type, case, narration, data, `prevHash`, `hash`. The hash is SHA-256 over the canonical JSON (sorted keys) of the entry including the previous hash. Changing any past entry breaks every link after it.

- The Security Centre recomputes the whole chain **in the user's browser** (Web Crypto) and reports the first broken entry, if any.
- `/api/ledger` exports the log with the verification result: this is the execution evidence.
- Anchors record the head hash at a moment in time, so someone with database access can't silently recompute the whole chain.

6.1 On-chain anchoring (Conflux eSpace testnet)

At each case milestone (case opened, payment approved, payment received or returned, refund received, institution verified, fraud flagged, case closed) the head hash is written to a public blockchain, where nobody, including us, can change it later.

- **Chain:** Conflux eSpace testnet (EVM, chain ID 71). A public chain built in China whose explorer, ConfluxScan, opens from mainland China. For a bank deployment the same contract runs on any EVM chain, or the anchor service writes to the bank's own consortium chain (e.g. FISCO BCOS).
- **Contract:** `CarryPayAnchor` (contracts/CarryPayAnchor.sol, about 20 lines). `anchor(head, seq, label)` emits `Anchored(index, head, seq, label, timestamp)`; only the deploying wallet may write. Deployed at `0x7cebbc3ac1a2ec61607025a589efa01b597c42cf` with the source verified on ConfluxScan. Compiled with solc 0.8.28 (`evmVersion: paris`) by `node scripts/anchor-contract.mjs compile`; the ABI and bytecode are committed in `src/server/anchor/artifact.json`.
- **What goes on-chain:** the 32-byte head hash, the entry number and a label such as `th-medical:payment received`. No names, amounts, accounts or case text. A SHA-256 head hash reveals nothing about the log, and it commits to every earlier entry, so one anchor covers the whole log up to that point.
- **Service** (`src/server/anchor/`): milestones from the log within 2.5 s share one anchor; transactions are sent one at a time from one wallet, retried on network errors, and confirmed; each anchor records its transaction, block and explorer link (`anchors.json`, the Security Centre, the `/api/ledger` export). It never blocks the agent or a payment, and it isn't an agent tool: the model can't call it and never sees the key.
- **Check:** "Check against the chain" in the Security Centre reads the transaction's `Anchored` event over RPC, compares it with the anchor, and recomputes the log up to that entry. After the Director's tamper demo the check fails with the entry that changed, while the chain still holds the original hash.
- **Key:** a testnet-only wallet created by `node scripts/anchor-key.mjs` straight into `.env.local` (never printed or committed). Without the key and contract the app works as before and anchors stay local.
- **What it proves, and what it doesn't:** it proves the log wasn't changed after an anchor, and when. It can't prove that what was logged was true, and an operator could still withhold the log entirely; that is what the hash chain, the bank's own records and the family's copies are for.

7. The public demo

- **Private sandbox per visitor.** With `CARRYPAY_SESSIONS=per-visitor`, `src/proxy.ts` gives each visitor a cookie and every API call works on that visitor's own world, so judges never drive each other's case. Idle worlds are removed; anchors from every world go through one send queue for the one wallet.
- **Story mode** (`/demo`): a recorded case unfolds chapter by chapter with captions; the judge approves the payments on the phone; a results card closes it. No AI cost.
- **Attack lab** (`/attack`): six attacks (changed bank details, a request for card and passport numbers, splitting across two cards, paying the doctor directly, buying FX in relatives' names, a duplicate deposit request). Each can be sent to the live agent, or the model can be *forced* to obey, to show the gateway blocking it with the rule cited.

- **Demo stage** (`/stage`), institution desk (`/hospital`), payment rails (`/bank`), bank officer view (`/bank/officer`), case pack (`/pack`) and director controls with chaos mode (`/director`).
- New visitors start in Replay mode, so the public link costs no AI credits until someone switches to Live.

8. Data and state

Sandbox state lives in a data folder (world snapshot, ledger as JSONL, anchors, recordings, passkeys), one folder per visitor world. In production this would be a database plus an append-only log store. Institutions are fictional; FX rates and fees are illustrative and labelled "sandbox" in the UI.

9. Testing and evaluation

`npm test` runs 88 tests in 14 files (the list below is the core; also: all four scenarios, uploaded bills, on-chain anchoring with a fake chain, private sessions, the attack lab, the MCP server, family answers, retrying dropped LLM connections):

- rules engine: tax history, pricing order, Pakistan caps, card limits, carried-cash rule, anti-splitting, SAFE quota and prohibited purposes, deadline-aware comparison
- every scenario end to end with the scripted agent, including the returned transfer, the scam email, the bill dispute, the refund and a tamper check, scored by the same evaluator as the live runs
- 18 red-team attacks on the safety layer (see SECURITY.md)
- sandbox institutions: multi-question replies, "final bill on discharge", transfer in flight, refund started once, follow-up skipping and cancelling
- privacy firewall: nothing personal in the masked snapshot, real values restored in tool inputs, Chinese names
- passkeys: a software P-256 authenticator signs approvals; changed amount, replay, missing user verification, wrong site, forged signature and unknown key are all rejected

Beyond unit tests:

- `node scripts/eval.mjs run <scenario> <n>` plays a scenario Live end to end N times and scores each run with `src/server/evaluate.ts` (task checks, safety checks, wake-ups, emails, tokens, AI cost, money saved vs paying by card). `report` writes EVAL.md and a chart. Current agent (v3.1): **43/45 runs passed every check**; the two failures were dropped connections to the model, with every task check passed. AI cost per case: $0.05 (admission checks) to $0.23 (the nine-day hospital case).
- `npm run redteam:llm` runs 20 attacks against the live model, with an ablation using a naive prompt (`REDTEAM_AGENT=naive`); see redteam/REPORT.md.
- `node scripts/evidence.mjs` writes evidence/: one log per scenario with its checks and on-chain anchors, each anchor matched against the chain.

10. Deployment

One long-running Node process (Next.js standalone build) in Docker behind Caddy with HTTPS, on a small VPS in Europe, at https://carrypay.duckdns.org; see DEPLOY.md. The server calls the LLM and the chain; the browser never does, and it never sees an API key or the anchor wallet's key.

11. Limitations

- Everything outside CarryPay is simulated: institutions, banks, the card network and FX rates. A pilot means connecting real bank and card APIs and a registry of verified institutions; the agent and gateway stay the same.
- Rules marked *reported* or *assumed* need confirming with a bank before production use; the UI shows each rule's status.
- Anchors prove the log wasn't changed after the fact, not that what was logged was true (§6.1).
- The live model can still make mistakes in wording or planning; the gateway limits what those mistakes can do, and EVAL.md lists every failure.